

DYAD FINANCE

**Diseno Guiado por Atributos de Calidad
(Attribute-Driven Design - ADD V3)**

Rol: Software Architect & Web3 Developer

Autor: Antigravity AI Partner

Fecha: Junio 2026

Version: 1.0 (Produccion)

1. Hacia una Arquitectura Rigurosa: Metodo ADD V3

El metodo ADD V3 (Attribute-Driven Design version 3), desarrollado por el Software Engineering Institute (SEI) de la Universidad Carnegie Mellon, es un enfoque sistematico para diseñar arquitecturas de software basándose en los Atributos de Calidad (ej. Seguridad, Rendimiento, Disponibilidad, Modificabilidad) en lugar de centrarse unicamente en los requisitos funcionales.

En protocolos DeFi complejos como Dyad Finance, donde se arriesgan fondos de usuarios y se depende de integraciones blockchain altamente volatiles, estructurar la arquitectura mediante ADD V3 garantiza que las decisiones de diseño mitiguen de forma explicita los riesgos tecnicos y operativos.

2. Atributos de Calidad Prioritarios (Drivers) para Dyad

Siguiendo el paso 2 de la metodologia ADD V3, definimos los escenarios de atributos de calidad criticos para el protocolo:

- Seguridad (Security): Evitar que un atacante manipule el balance del pool de RWA o la logica de plusvalia. Escenario QAS-1: Un perito malicioso envia un reporte de precio falso. El oraculo on-chain detecta la anomalia mediante firmas criptograficas y cancela la transaccion en menos de 1 bloque.
- Rendimiento (Performance): Escenario QAS-2: Bajo alta carga de animaciones en la dApp y multiples peticiones RPC, el contador de mineria RWA acumulado debe actualizarse en tiempo real a 60 FPS sin congelar la UI ni saturar el hilo principal del navegador.
- Disponibilidad (Availability): Escenario QAS-3: Si el nodo RPC de Sepolia/Ethereum (ej. Alchemy) se desconecta temporalmente, la dApp debe degradar su funcionamiento de forma graciosa mostrando datos cacheados y reconectandose en menos de 3 segundos.
- Modificabilidad (Modifiability): Escenario QAS-4: El equipo financiero decide cambiar el algoritmo del token volatil apalancado (de un modelo basico a un ERC4626 puro). La modificacion del contrato y del frontend debe completarse en menos de 2 dias de desarrollo sin alterar los modulos de bienes raices (RWA) o swap.

3. Conceptos de Diseño y Tácticas Arquitectónicas Seleccionadas

Para dar respuesta a los drivers prioritarios identificados (ADD Paso 3), aplicamos las siguientes tácticas de arquitectura de software:

A. Tácticas para Seguridad (Security):

- Verificar Origen del Mensaje: Uso de firmas ECDSA criptográficas en los reportes de tasación enviados desde el oráculo para asegurar la autenticidad antes de acuñar usdJ.
- Limitación de Privilegios: Control de acceso basado en roles (RBAC) en los métodos de tasación de VaultV2, restringiendo la manipulación de la plusvalía RWA a perfiles autorizados.

B. Tácticas para Rendimiento (Performance):

- Fail-Safe de Red (Timeout): Implementar límites de tiempo (timeouts) de 800ms en el preloader para peticiones masivas de recursos, evitando congelamientos de hilo en dispositivos móviles.
- Procesamiento en Lotes (Batching): Agrupar lecturas de contratos inteligentes usando Multicall para consolidar balances de ETH, usdJ, jETH, USDC y LINK en una sola llamada de red.

C. Tácticas para Disponibilidad (Availability):

- Degradación Gradual (Graceful Degradation): Al perder conexión con el nodo blockchain, el sistema desactiva temporalmente los botones de transacción y muestra los últimos saldos almacenados en la cache local (`localStorage` / memory cache) con una etiqueta de advertencia.

D. Tácticas para Modificabilidad (Modifiability):

- Abstracción e Interfaz: Aislar la lógica de negocio pura (Dominio) usando puertos e interfaces independientes de la tecnología Web3, aplicando DDD y CQRS.

4. Instanciación de Elementos de Diseño y Responsabilidades

Asignamos responsabilidades concretas a los elementos del sistema basándonos en las tácticas elegidas (ADD Paso 4):

1. RealEstateOracle (Contrato & Servicio): Responsable exclusivo de mantener la plusvalía de propiedades firmada criptográficamente. Actúa como guardian de la seguridad.
2. VaultV2 (Contrato & Adaptador): Responsable del cálculo matemático de deuda colateral y de la acuñación y quemado de usdJ y jETH. Aplica límites estrictos de LTV (75%).
3. StakingPool (Contrato & Adaptador): Habilita la lógica de emisión y rendimiento de jKERO.
4. CacheService (UI/Infraestructura): Almacena y sincroniza localmente balances, precios y reportes. Es el garante de la Disponibilidad del sistema ante cortes de red.
5. CanvasAnimator (UI / Animación): Se encarga del bucle de renderizado interactivo (60 FPS) de las imágenes de desensamblado 3D sin interrumpir las llamadas de negocio, garantizando el rendimiento visual.

5. Conclusion del Análisis ADD V3

El uso de ADD V3 nos ha permitido ir más allá de la simple programación de funcionalidades de usuario. Al priorizar explícitamente la Seguridad, el Rendimiento, la Disponibilidad y la Modificabilidad, Dyad Finance se convierte en un sistema resistente a fallas de infraestructura de nodos, blindado contra ataques básicos de tasación y preparado para cambiar sus contratos inteligentes con el menor impacto en el código del frontend.

Los siguientes pasos de refinamiento metodológico involucran la definición formal de los contratos de interfaz (APIs) y esquemas de datos de los adaptadores de infraestructura.